

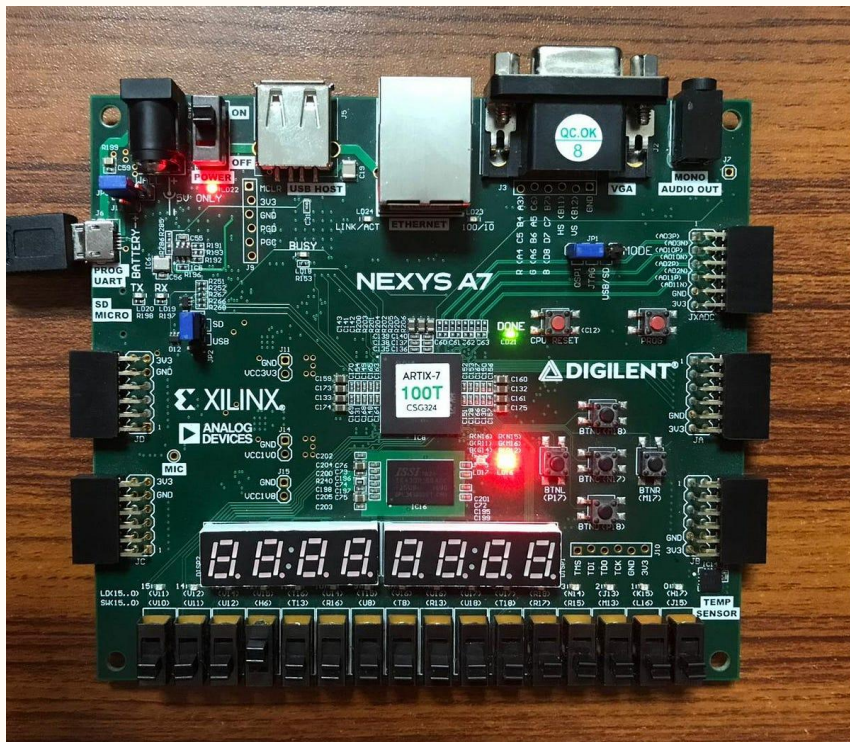


FEXO

UART-based Memory Controller

Muhammad Farhan Shakeel

What We're Building



Core Goal

Build a memory controller that communicates with a PC via UART, reading and writing a synchronous RAM using FIFOs and a finite state machine.

- Uart receive serial data from the Laptop and store it in rx_fifo.
- This data is stored in the Memory we made async_ram.
- When read command is received from a specific memory location, that data must return back from uart_transmitter to laptop serially.

Target Hardware

- Nexys-7 / Basys-3 — Artix-7 XC7A100T (xc7a100tcsg324-1)
- 100 MHz system clock
- PMOD USB-UART for serial link to host PC

Working Procedure



The host PC sends ASCII command packets over UART. The RX FIFO decouples the serial interface from the controller FSM. The TX FIFO buffers read responses before transmission back to the host.

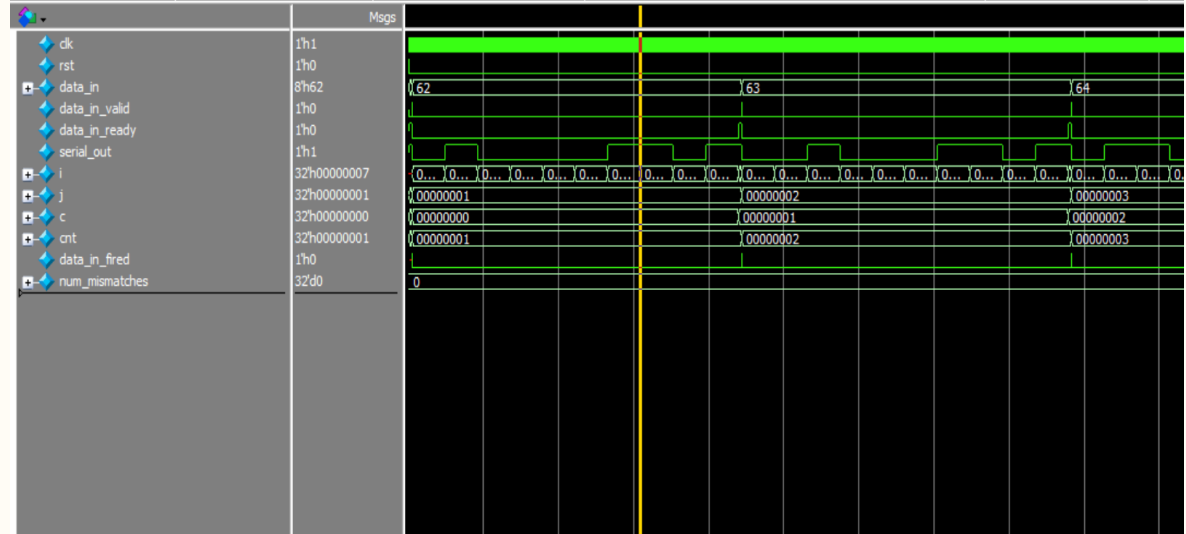
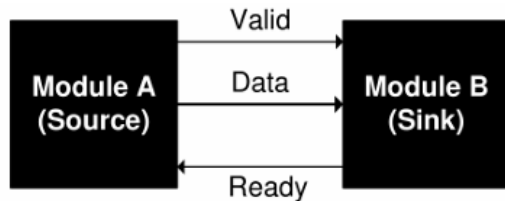
Serial Communication at 115 200 Baud

Module Parameters

- System clock: **100 MHz**
- Baud rate: **115 200**
- Separate TX and RX sub-modules
- Handshake interface: **valid / ready**

UART Frame Structure

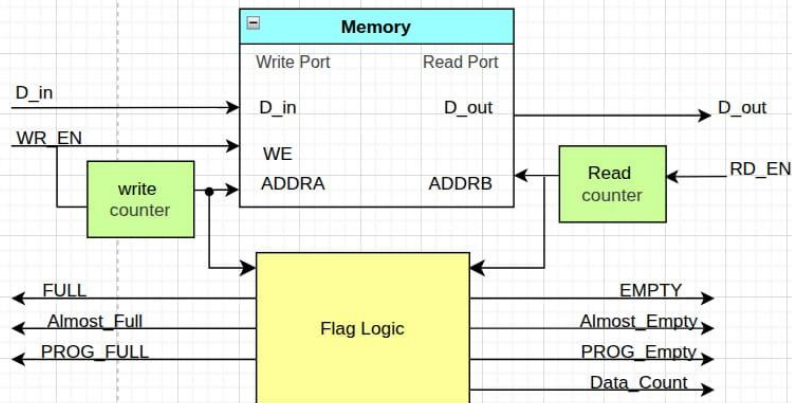
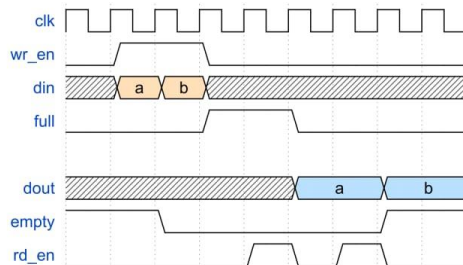
- 1 start bit (logic 0)
- 8 data bits, LSB first
- 1 stop bit (logic 1)
- Total: 10 bits per character



Asynchronous 8×32 FIFO with Zero-Cycle Read Latency

Key Design Features

- Depth: **32 entries**, Width: **8 bits**
- Uses ASYNC_RAM from EECS151 library
- **0-cycle read latency** — data appears on **dout** the same cycle as **rd_en**
- Full/empty flags via pointer comparison with extended MSB
- Overflow and underflow protection built in

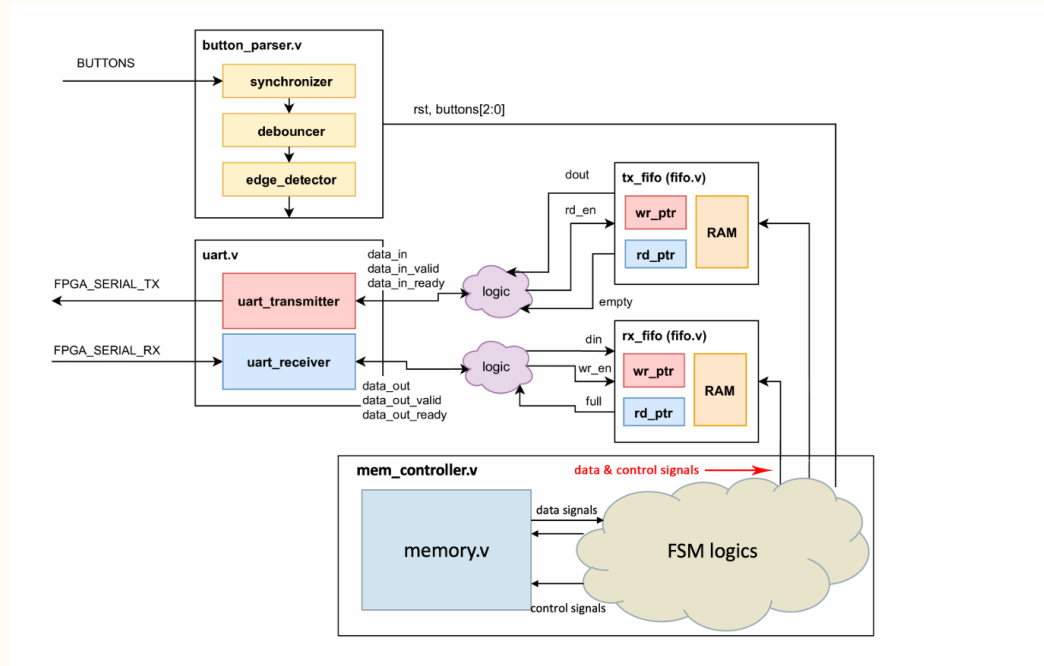


Common Clock FIFO

Memory Controller — 7-State FSM

The FSM is written for QuestaSim compatibility.
States transition based on FIFO status flags and
UART input.

- **IDLE** — Wait for incoming command
- **GET_CMD** — Read command byte ('0' or '1')
- **GET_ADDR** — Read memory address
- **GET_DATA** — Read write data byte
- **DO_WRITE** — Write to RAM
- **DO_READ** — Read from RAM
- **SEND_READ** — Transmit result via TX FIFO



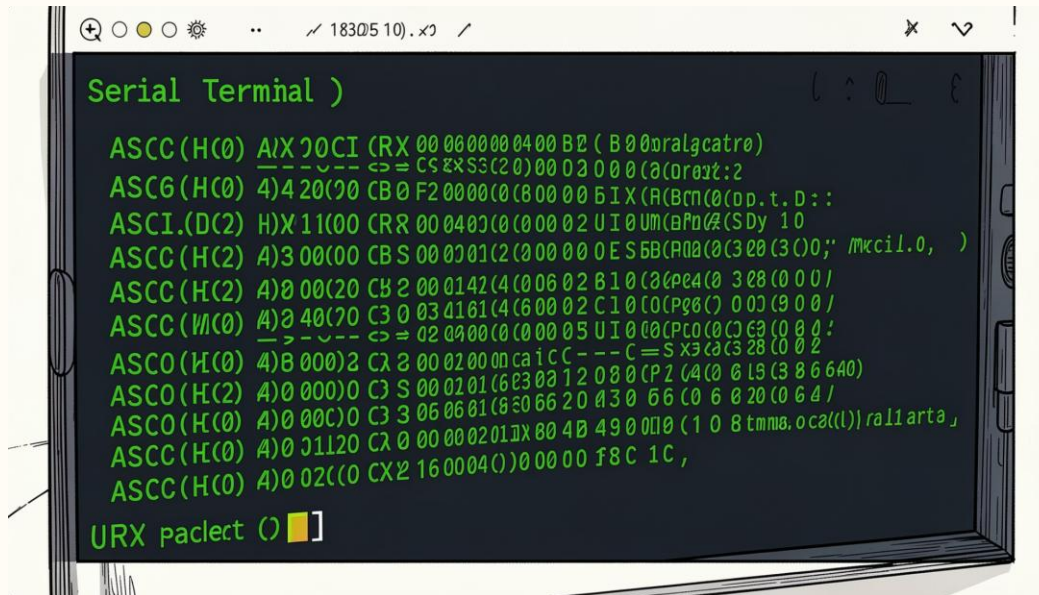
Read & Write Command Packets

Write Packet — 3 Bytes

Byte	Meaning
1	Write command
A	Target address
2	Data to store

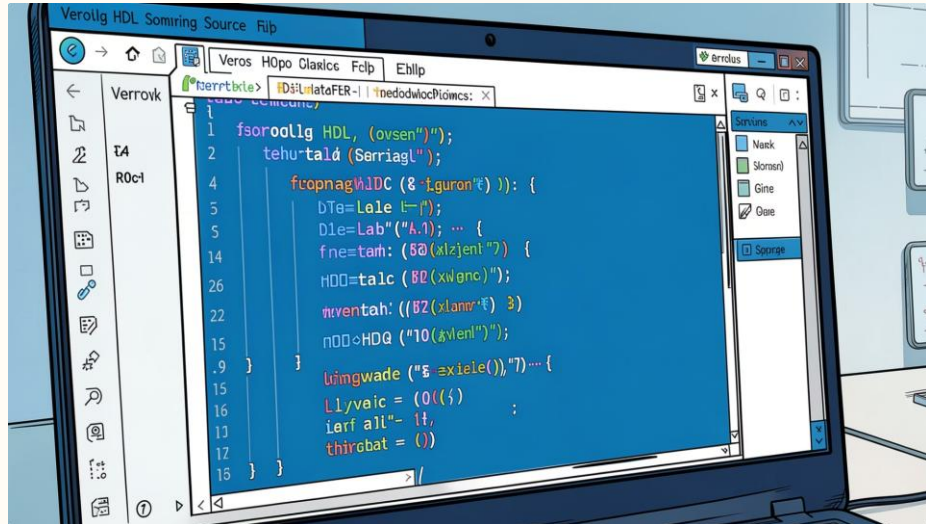
Read Packet — 2 Bytes

Byte	Value	Meaning
0		Read command
A		Target address



FSM Logic for “READ” & “Write”

1. 1 command is received and memory FSM moves to Write state.
2. Next command is taken as location in for the data to be stored.
3. 3rd entered value is taken as data and it get stored in memory.
4. When “0” is pressed, FSM logic detects it and entered in the READ state.
5. After “0” next command is taken as memory addr, and it returns the data stored there.



FPQA Deployment, Challenges & Conclusion

Board Setup

- Nexys-7 / Basys-3 — Artix-7 XC7A100T, 100 MHz
- PMOD USB-UART on PMOD port
- `BTN[0]` = reset

Hardware Test

1. Program FPGA via Vivado
2. Connect PMOD USB-UART to PC
3. Open screen `/dev/ttyUSB0 115200`
4. Write: `1 + A (addr 65) + B (data 66)`
5. Read: `0 + A` → terminal shows `B`

Delivered

UART-controlled memory system verified in simulation and on FPGA. Portable, parameterized design.